

```

# %% Imports and File Paths
import pandas as pd
import w6_sol_functions as fn

# File paths to the toy_net data
# (update these if your data files are in a different
location).
node_file = "data/toy_net/nodes.csv"
link_file = "data/toy_net/net.csv"
demand_file = "data/toy_net/trips.csv"

# Load CSV files into Pandas DataFrames
df_nodes = pd.read_csv(node_file, sep=",")
df_links = pd.read_csv(link_file, sep=",")
df_ods = pd.read_csv(demand_file, sep=",")

# Set index using node IDs for efficient lookups
df_nodes.set_index("node", inplace=True, drop=False)
df_links.set_index(["init_node", "term_node"],
inplace=True, drop=False)
df_ods.set_index(["orig", "dest"], inplace=True,
drop=False)

# Construct the forward star dictionary to store the
downstream nodes for each node
forward_star = {}
for row in df_nodes.itertuples(index=False):
    forward_star[row.node] = []
for row in df_links.itertuples(index=False):
    forward_star[row.init_node].append(row.term_node)

# %% Traffic Assignment Parameters

max_iter = 10000 # Maximum number of iterations
target_rg = 0.0001 # Target threshold value for the
Relative Gap (RG)
target_aec = 0.0001 # Target threshold value for the
Average Excess Cost (AEC)

# %% Initialisation

# Create the 'cost' column in df_links to store link
travel time;
# Initialise the values to 'free_flow_time'
df_links["cost"] = df_links["free_flow_time"]

# Create the 'flow' column in df_links to store link
flow values;
# Initialise with 0.0
df_links["flow"] = 0.0

# Create the 'aon_flow' column to store All-or-
Nothing (AON) assignment flows;
# Used as 'target' link flow (x*) for flow update: x
<- lambda*x* + (1-lambda)*x
df_links["aon_flow"] = 0.0

# Create the 'shortest_path_time' column in df_ods;
# Initialise with 0.0
df_ods["shortest_path_time"] = 0.0

# Create the 'shortest_path' column in df_ods;
# Initialise with None to store object (list) type
df_ods["shortest_path"] = None

# Perform All-or-Nothing Assignment to get initial
link flows
# Input: df_links['cost'], df_ods['demand']; Output:
df_links['aon_flow']
fn.allOrNothingAssignment(df_nodes, df_links, df_ods,
forward_star)

# Use All-or-Nothing link flows as initial 'current'
link flows
df_links["flow"] = df_links["aon_flow"]

# Update link costs (link travel times)
# Input: df_links[['flow', 'capacity',
'free_flow_time', 'b', 'power']]
# Output: df_links['cost']
fn.updateLinkCost(df_links)

print("Initial AON Assignment Results:")
print(df_links[["cost", "flow"]])

# %% ===== Main Iterative Loop for UE Traffic
Assignment =====

step_size = 1 # Initialise the step size (lambda) as
1
iteration = 0 # Initialise the iteration counter

while iteration <= max_iter:

    # Update OD shortest path times based on current
link costs
    # Input: df_links['cost']
    # Output: df_ods[['shortest_path_time',
'shortest_path']]
    fn.updateOdShortestPathTime(df_nodes, df_links,
df_ods, forward_star)

    # Calculate convergence measures
    # Input: df_links[['flow', 'cost']],
df_ods[['demand', 'shortest_path_time']]
    # Output: relative_gap, avg_excess_cost
    relative_gap, avg_excess_cost =
fn.calculateConvergenceMeasures(df_links, df_ods)

    # Display iteration details to monitor the
convergence trend
    print(
        f"Iter={iteration}, Step_size=
{step_size:.5f}, "
        f"RG={relative_gap:.5f}, AEC=
{avg_excess_cost:.5f}"
    )

    # Terminate if both convergence criteria are met
    if relative_gap < target_rg and avg_excess_cost <
target_aec:
        break

    # Terminate if maximum iterations are reached;
    # Why do we need this even if the while-loop
condition is already set to "iteration <= max_iter"?
    # This is to terminate the loop immediately after
reaching max_iter, without performing another
iteration, so that the final assignment results are
from the max_iter-th iteration.
    # For example, we can control how many iterations
to perform by setting max_iter = 10, and we want the
final assignment results to be from the 10-th
iteration.
    # If we don't have this, the loop continues until
the end, where iteration is incremented to "max_iter
+ 1" and another traffic assignment iteration is
performed.
    # Then, the final assignment results will be from
the (max_iter + 1)-th iteration, which is not what we
want.
    if iteration == max_iter:
        break

    # Increment iteration counter
    iteration += 1

    # Get target link flows (x*) by performing AON
assignment
    # Input: df_links['cost'], df_ods['demand'];
Output: df_links['aon_flow']
    fn.allOrNothingAssignment(df_nodes, df_links,
df_ods, forward_star)

    # Compute MSA step size: lambda = 1 / (n + 1)
    step_size = 1 / (iteration + 1)

    # Shift link flows: x <- lambda * x* + (1 -
lambda) * x
    # Input: step_size, df_links[['flow',
'aon_flow']]; Output: df_links['flow']
    fn.shiftLinkFlows(step_size, df_links)

    # Update link costs
    # Input: df_links[['flow', 'capacity',
'free_flow_time', 'b', 'power']]
    # Output: df_links['cost']
    fn.updateLinkCost(df_links)

# Display the final UE assignment results
print("UE Assignment Results:")
print(df_links[["cost", "flow"]])

```